

API documentation

REST + device protocol reference for the TRUSTMESH backend (Fastify, <http://localhost:4000>).

Generated 2026-09-29 from the repository · local-first hackathon prototype

Contents

1. API guide: authentication, endpoints, error codes
2. Device protocol: HMAC envelopes, sessions, replay rules
3. Endpoint reference generated from OpenAPI

API guide

Base: <http://localhost:4000> (browser uses relative `/api/...` through the Vite proxy). Live OpenAPI UI: </docs> ; exported spec: [openapi.json](#). All errors: `{ "error": CODE, "message": text }`.

Auth: session cookie `tm_sid` (HttpOnly, SameSite=Strict, 12 h). Every non-GET session request (except `/auth/*` and `/device/*`) must send `x-csrf-token` (returned by `/auth/me` or login). Device endpoints use HMAC envelopes only.

Method & path	Who	Purpose
GET <code>/health</code> · <code>/ready</code>	public	liveness · readiness (503 when chain/manifest/fingerprint not OK)
GET <code>/api/v1/auth/personas</code>	public	local dev identities (only when dev signer enabled)
POST <code>/api/v1/auth/dev-login</code> <code>{persona}</code>	public (dev only)	session for a dev identity
POST <code>/api/v1/auth/nonce</code> <code>{address}</code> → <code>{message}</code>	public	SIWE-style message (domain, chain id, nonce, 5 min expiry)
POST <code>/api/v1/auth/wallet</code> <code>{message, signature}</code>	public	verifies domain, chain, expiry, single-use nonce, signature
GET <code>/api/v1/auth/me</code> · POST <code>/auth/logout</code>	session	
POST <code>/api/v1/device/{hello,teleme try,poll,ack}</code>	device (HMAC)	protocol — see IOT_INTEGRATION.md
GET <code>/api/v1/stream</code>	public	SSE (<code>Last-Event-ID</code> or <code>?last=</code> replay; <code>resync</code> after server restart)
GET <code>/api/v1/chain/manifest</code> · <code>/chain/status</code>	public	deployment manifest · RPC/indexer health

Method & path	Who	Purpose
GET /api/v1/chain/txs · /chain/events · /chain/tx/:hash	session	local explorer (receipt read from node)
POST /api/v1/chain/track {hash}	session	record a browser-wallet tx sent by the signed-in wallet
POST /api/v1/chain/reindex	admin	reset indexer cursor
GET /api/v1/overview	session	dashboard counts
GET/POST /api/v1/assets · GET /assets/:id	session · registrar	inventory / register on-chain / passport
POST /api/v1/assets/:id/enroll- tag	owner/admin	enroll tag last read by a bound device (≤30 s)
PUT /api/v1/assets/:id/sensor- policy	owner/admin	versioned, audited sensor thresholds
GET /api/v1/public/assets/:id	public	redacted read-only passport
GET /api/v1/transfers · POST /transfers/:id/challenge	session · owner/recipient	list · reissue challenge
POST /api/v1/actions/:name	dev session	whitelisted lifecycle action signed by the session's dev key (USE_WALLET for wallets)
POST /api/v1/incidents/:id/report s · /maintenance-evidence	session / assigned technician	report hash · anchor fresh nominal maintenance evidence
GET /api/v1/incidents	session	incidents, reports, derived technician record
GET/POST /api/v1/devices · /devices/:id/telemetry · POST /devices/:id/{bind, revoke, ro tate-key}	session · admin	devices, telemetry, provisioning
GET/POST /api/v1/commands	session · on-chain owner/custodian	actuator commands
GET/POST /api/v1/credentials · POST /credentials/revoke	session · issuer	on-chain credentials
GET/POST /api/v1/organizations · POST /organizations/members	session · admin	orgs
GET /api/v1/evidence · /integrity/:eventId	session	evidence list · verification against chain
POST /api/v1/integrity/sandbox · /sandbox/:id/{modify, restore }	session	tamper demo on copies only

Method & path	Who	Purpose
GET /api/v1/audit · /diagnostics	session	audit log · jobs, sessions, LAN URLs
GET /api/v1/public/kit	public	real-kit (ESP32-017) status: online, transport, firmware profile, per-sensor health — drives the "good to go" celebration
GET /api/v1/demo/state[? asset=PUMP-017]	session	plain-English passport for the Live Demo (owner, holder, health, locked, sensor online)
POST /api/v1/demo/{reset,handover,alarm,trySell,repair,tamper}[?asset=PUMP-017]	session, dev mode, loopback	guided story: performs the real on-chain actions as owner/technician/inspector; returns a plain-English log with tx hashes. <code>HARDWARE_OFFLINE</code> if the real kit isn't connected
GET/POST /api/v1/sim	session, dev mode	in-browser SIMULATED device: status, start/stop, patch sensor values (range-checked)

Actions (/actions/:name , same map used by wallets — packages/shared/src/actions.ts):

activateAsset, retireAsset, setPolicy, requestTransfer, acceptTransfer, cancelTransfer, expireTransfer, completeTransfer, acknowledgeIncident, assignMaintenance, startMaintenance, submitMaintenance, assignInspector, completeInspection .

Notable error codes: UNAUTHENTICATED, FORBIDDEN, CSRF, DEV_SIGNER_DISABLED, USE_WALLET, CHAIN_UNAVAILABLE, CONTRACT_<SolidityError> (e.g. CONTRACT_AssetBlocked , CONTRACT_SelfInspection), TX_REVERTED, NOT_OPERATIONALLY_AUTHORIZED, BAD_PARAMS, NO_FRESH_EVIDENCE, ANOMALY_PRESENT, NO_TAG_SEEN ; device: BAD_ENVELOPE, UNKNOWN_DEVICE, DEVICE_REVOKED, KEY_VERSION_MISMATCH, BAD_MAC, BAD_BASE64, PAYLOAD_TOO_LARGE, UNSUPPORTED_SCHEMA, BAD_SCHEMA, SESSION_UNKNOWN, SESSION_CLOSED, SESSION_REPLAY, SEQ_REPLAY, CONFLICTING_DUPLICATE, BINDING_MISMATCH, BINDING_VERSION_STALE, RATE_LIMITED .

Device protocol

Envelope (both transports)

```
{ "deviceId": "ESP32-017", "keyVersion": 1, "payloadB64": "<base64 of exact UTF-8 payload bytes>", "macHex": "<hmac-sha256 hex>" }
```

MAC = HMAC-SHA256(key = 32-byte per-device secret, message = preimage):

```
domain (4 ASCII bytes: "TMD1" device→server, "TMS1" server→device)
u8 len(deviceId) || deviceId (ASCII)
u32be keyVersion
u32be len(payload) || payload bytes
```

The backend verifies the MAC over the decoded bytes **before** parsing JSON; payloads > 4096 bytes are rejected. Server responses are sealed with `TMS1` and verified by the firmware (commands and challenges are therefore authenticated too). Shared vectors: `packages/shared/test-vectors.json`, checked by `npm run test:shared` and by the firmware host test `pio test -e native` (same C++ used on the ESP32).

This is **symmetric** authentication: the backend holds the key and could forge a message. It is not independent device attestation.

Payload v1 (authenticated)

field	meaning
<code>v</code>	schema version (only 1 accepted)
<code>type</code>	<code>hello</code> · <code>telemetry</code> · <code>poll</code> · <code>ack</code>
<code>deviceId</code> , <code>bootId</code>	identity; <code>bootId</code> random per boot
<code>sessionId</code>	from the authenticated hello response
<code>seq</code>	monotonically increasing for the boot
<code>eventId</code>	<code><bootId>-<seq></code> ; stable across retries
<code>uptimeMs</code> , <code>ts?</code>	device time; server receipt time is recorded separately
<code>bindingVersion</code> , <code>profile</code> , <code>caps</code>	declared configuration (binding checked against registry)
<code>r / q</code>	readings (explicit keys, see <code>READINGS</code> in <code>shared/constants.ts</code>) and per-reading quality (<code>ok</code> , <code>warmup</code> , <code>fault</code> , <code>absent</code> , <code>nofix</code> , <code>uncal</code>)
<code>challengeId?</code>	active custody challenge (from poll)
<code>local</code>	<code>pumpOn</code> , <code>interlock</code> , <code>manualStop</code> , <code>buffered</code> , <code>dropped</code>
<code>cmd</code>	(ack) `{commandId, status: applied`

Provenance (REAL/SIMULATED) is **never** read from the payload: it comes from the device's provisioning (DB + DeviceRegistry).

Sessions, replay, duplicates

- `hello` needs a random hex `nonce`; `(device, bootId, nonce)` is unique → replayed hello = 409 `SESSION_REPLAY`. A new hello closes the previous session.
- Telemetry must reference a known session. Event id is unique per device: an identical retry returns the original (MAC'd) result with `duplicate:true` and creates nothing new; same id + different bytes = 409 `CONFLICTING_DUPLICATE`; same `(session, seq)` with another id = 409 `SEQ_REPLAY`.
- Samples from a closed session, out of order, or >30 s old are stored as **delayed** and never satisfy a challenge.
- A payload `assetId` that differs from the registered binding → 409 `BINDING_MISMATCH`; stale `bindingVersion` → 409 `BINDING_VERSION_STALE` (device re-hellos). Rebinding invalidates outstanding challenges and closes sessions.
- Rate limit: 15 req/s sustained, burst 30 per device. Body limit 8 KB.

Canonical evidence (what gets hashed)

Schema `trustmesh.evidence/1`: `{schema, eventId, kind, deviceId, assetId, bindingVersion, provenance, profile, sessionId, seq, observedAt, policyVersion, sensorPolicyVersion, challengeId, flags, anomalies[], payloadSha256, readings{key:{v,unit,q}}}`. Readings are fixed-point integers (°C×100, mm, raw ADC, degrees×1e7...). Canonical bytes = JSON with recursively sorted keys, no whitespace, floats rejected. Hash = `keccak256(utf8(canonical))`; the hash and receipts are not part of the preimage. The raw RFID UID never enters canonical evidence — only `tagref = sha256("trustmesh-tag:"+UID)[0:16]`.

Transports

- **Wi-Fi**: firmware POSTs to `TM_BACKEND_URL` (laptop LAN IP, never localhost). API must listen on `0.0.0.0` (`API_HOST=0.0.0.0` in `.env`).
- **USB serial**: firmware prints `{"t":"req","id":N,"path":"/api/v1/device/...","body":{"envelope}}` lines; `npm run bridge -- --port <port>` POSTs them and writes `{"t":"res","id":N,"status":S,"body":{"..."}}`. Log lines start with `#`. The bridge announces itself with `{"t":"bridge","v":1}` every 2 s, forwards only the four device paths, and relays `!...` diagnostic lines you type.

Commands

Delivered in the MAC'd poll response: `{commandId, action, params, deviceId, assetId, bindingVersion, issuedAt, expiresAt (server s), txRef}`; TTL 60 s. Firmware re-validates device/asset/binding/expiry (server-time offset from each authenticated response — no NTP needed), dedupes the last 16 ids in NVS, marks a command `pending` in NVS before physical action and acks `uncertain` after a reset mid-action (never repeats it). Actions: `GATE_OPEN` (custodian only, references the custody tx), `GATE_CLOSE`, `PUMP_ON` (1–30 s, requires on-chain NORMAL/no incident/AVAILABLE|IN_CUSTODY), `PUMP_OFF` (always allowed), `INDICATE`, `CLEAR_LOCAL_STOP`. UI shows created → delivered → acknowledged separately; acknowledgement is not observed physical effect.

Provision / bind / revoke / rotate

UI → Devices (admin) or `POST /api/v1/devices` → secret shown once → put in `firmware/include/secrets.h`. Seeded devices' secrets live in `.local/devices.json` (git-ignored, kept across demo resets). `POST /devices/:id/bind` (new binding version), `/revoke` (on-chain + key retired), `/rotate-key` (new secret + on-chain keyVersion).

Simulator

`npm run demo:simulate` uses the SIMULATED identity `SIM-ESP32-017` bound to `SIM-PUMP-017` through the same HTTP protocol. Scenarios: `live` (interactive keys), `custody`, `incident`, `integrity`, `full`. Real-hardware policies reject its evidence (tested).

Endpoint reference

Generated from `docs/openapi.json` (OpenAPI 3.0.3, 59 operations). Live interactive version: <http://localhost:4000/docs>.

Health

GET `/health`

Liveness check. Always 200 while the API process runs.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET `/ready`

Readiness: database, chain RPC, manifest and deployment fingerprint. 503 when the chain is unavailable or mismatched.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Authentication

GET `/api/v1/auth/personas`

Lists the local development identities (only when the dev signer is enabled).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST `/api/v1/auth/dev-login`

Signs in as a local development identity (public Hardhat test keys; chain 31337 only; loopback only). Returns the CSRF token.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "persona"
  ],
  "properties": {
    "persona": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/auth/nonce`

Creates a single-use sign-in message (domain, chain id, nonce, 5-minute expiry) for a wallet address.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "address"
  ],
  "properties": {
    "address": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/auth/wallet`

Verifies a signed sign-in message (domain, chain, expiry, unused nonce, signature) and starts a wallet session.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "message",
    "signature"
  ],
  "properties": {
    "message": {
      "type": "string",
      "maxLength": 2000
    },
    "signature": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/auth/me`

Current session user (name, address, roles, CSRF token) and whether the dev signer is enabled.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/auth/logout`

Ends the browser session.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Public (no sign-in)

GET`/api/v1/public/assets/{id}`

Public read-only passport (what the QR code opens): no secrets, raw tag ids or private data.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/public/kit`

Public real-kit status (ESP32-017): online, transport, firmware profile and per-sensor health.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Device protocol (HMAC envelopes)

POST`/api/v1/device/hello`

Device → server (HMAC envelope with a random nonce): opens a session; the MAC'd response carries the session id, server time, asset binding and poll interval.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST**/api/v1/device/telemetry**

Device → server (HMAC envelope): one signed sensor sample. Verified, deduplicated, evaluated against policy; may create evidence and chain transactions.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/device/poll`

Device → server (HMAC envelope): fetches pending signed commands and the active custody challenge.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/device/ack`

Device → server (HMAC envelope): acknowledges a command as applied, rejected or uncertain.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Live demo & simulator

GET`/api/v1/demo/state`

Plain-English Live Demo passport (owner, holder, health, locked, sensor online). Add `?asset=PUMP-017` for the real kit.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/demo/{step}`

Runs one Live Demo story step — reset, handover, alarm, trySell, repair, tamper — performing the real on-chain actions as owner/technician/inspector; returns a plain-English log with transaction hashes. ? asset=PUMP-017 uses the real kit (HARDWARE_OFFLINE if it isn't connected). Dev mode, loopback only.

Parameter	In	Required	Type
step	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/sim`

Status of the in-browser SIMULATED device (dev mode).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/sim`

Starts/stops the SIMULATED device or patches its sensor values (range-checked, audited).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Assets

GET`/api/v1/assets`

Inventory: every asset with owner, custodian, location, lifecycle, condition and policy, plus an address→name map.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST**/api/v1/assets**

Registers a new asset on-chain (registrar/admin). The owner must then activate it with their own transaction.

REQUEST BODY (JSON SCHEMA)

```

{
  "type": "object",
  "required": [
    "id",
    "name",
    "location",
    "owner",
    "requireReal"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "name": {
      "type": "string",
      "minLength": 2,
      "maxLength": 80
    },
    "model": {
      "type": "string",
      "maxLength": 80
    },
    "description": {
      "type": "string",
      "maxLength": 500
    },
    "location": {
      "type": "string",
      "maxLength": 80
    },
    "owner": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "custodian": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "requireReal": {
      "type": "boolean"
    },
    "requiredFlags": {
      "type": "integer",
      "minimum": 0
    },
    "maxEvidenceAge": {
      "type": "integer",
      "minimum": 10,
      "maximum": 86400
    }
  }
}

```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/assets/{id}`

Full passport: asset, bound devices with latest readings, transfers, incidents, evidence, bindings, policies, reports and confirmed chain events.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/assets/{id}/enroll-tag`

Enrolls the RFID tag most recently read (≤ 30 s) by a bound device as the asset's identity tag. Owner or admin.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

PUT`/api/v1/assets/{id}/sensor-policy`

Updates the versioned, audited sensor-interpretation thresholds (a policy change, not a sensor change). Owner or admin.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Transfers & lifecycle actions

GET`/api/v1/transfers`

Transfers with their challenges (status, expiry, bindings, last verification result).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/transfers/{id}/challenge`

Owner or recipient: issues a fresh custody challenge (invalidates the previous one).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/actions/{name}`

Performs a whitelisted lifecycle action (e.g. requestTransfer, acceptTransfer, assignMaintenance, completeInspection) as the signed-in development identity; returns the confirmed transaction hash. Wallet sessions get USE_WALLET and sign in the browser instead. Loopback only.

Parameter	In	Required	Type
name	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Incidents, maintenance, inspection

POST`/api/v1/incidents/{id}/reports`

Stores a maintenance or inspection report and returns the hash that gets committed on-chain.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/incidents/{id}/maintenance-evidence`

Assigned technician: anchors fresh, anomaly-free evidence from a live bound device and returns its event id (used for wallet flows).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/incidents`

Incidents with status, technician, inspector, reports and the derived technician record.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Devices & commands

GET`/api/v1/devices`

All devices with provenance, profile, capabilities, binding, last authentication result and latest readings.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/devices`

Admin: provisions a device on-chain (provenance fixed) and returns its secret key ONCE.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "id",
    "provenance",
    "profile"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "label": {
      "type": "string",
      "maxLength": 80
    },
    "provenance": {
      "enum": [
        "REAL",
        "SIMULATED"
      ]
    },
    "profile": {
      "type": "string",
      "maxLength": 32
    },
    "caps": {
      "type": "integer",
      "minimum": 0
    },
    "assetId": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/devices/{id}/telemetry`

Recent authenticated samples from one device with flags, anomalies and delayed markers.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/devices/{id}/bind`

Admin: binds a device to an asset (new on-chain binding version; closes sessions; invalidates open challenges).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/devices/{id}/revoke`

Admin: revokes a device on-chain; its evidence is rejected from then on.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/devices/{id}/rotate-key`

Admin: rotates the device key (on-chain key version) and returns the new secret ONCE.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/commands`

Actuator command log: created → delivered → acknowledged / rejected / expired / uncertain.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/commands`

Creates a signed, 60-second actuator command (GATE_OPEN custodian-only; PUMP_ON needs on-chain NORMAL, 1–30 s; PUMP_OFF always allowed).

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "action"
  ],
  "properties": {
    "deviceId": {
      "type": "string"
    },
    "action": {
      "type": "string"
    },
    "params": {
      "type": "object"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Evidence & integrity

GET`/api/v1/evidence`

Evidence commitments (canonical evidence hashes) with pipeline status and transaction.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/integrity/{eventId}`

Verifies one evidence record: recomputes keccak256 of the canonical bytes and compares with the on-chain commitment (MATCH / MISMATCH / COMMITMENT_ABSENT / UNVERIFIABLE_CHAIN_UNAVAILABLE / SCHEMA_UNSUPPORTED).

Parameter	In	Required	Type
eventId	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/integrity/sandbox`

Creates a sandbox copy of an evidence record for the tamper demonstration and verifies it.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/integrity/sandbox/{id}/modify`

Changes one field of the sandbox copy (optionally also forging its cached hash) and re-verifies against the chain.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/integrity/sandbox/{id}/restore`

Restores the sandbox copy from the original and re-verifies.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Credentials & organisations

GET`/api/v1/credentials`

Technician and inspector credentials with issue/expiry/revocation state.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/credentials`

Issuer/admin: issues an on-chain TECHNICIAN or INSPECTOR credential valid for N days.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "holder",
    "role",
    "days"
  ],
  "properties": {
    "holder": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "role": {
      "enum": [
        "TECHNICIAN",
        "INSPECTOR"
      ]
    },
    "days": {
      "type": "number",
      "exclusiveMinimum": 0,
      "maximum": 3650
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/credentials/revoke`

Issuer/admin: revokes an on-chain credential.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/organizations`

Organisations and their members.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/organizations`

Admin: creates an organisation.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "name",
    "kind"
  ],
  "properties": {
    "name": {
      "type": "string",
      "minLength": 2,
      "maxLength": 80
    },
    "kind": {
      "type": "string",
      "maxLength": 32
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/organizations/members`

Admin: adds or moves a wallet address into an organisation.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Blockchain

GET`/api/v1/chain/manifest`

Deployment manifest: chain id, contract addresses, ABI version, deployment block, fingerprint.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/chain/status`

RPC health, latest block, fingerprint, dev-signer state and indexer cursor.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/chain/txs`

Transactions submitted by the backend or tracked for wallets, with receipt status.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/chain/events`

Decoded contract events indexed from the current deployment (optionally filtered by name).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/chain/tx/{hash}`

Transaction detail read from the node: receipt, block, gas, decoded events and related evidence (the local explorer).

Parameter	In	Required	Type
hash	path	yes	string

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/chain/reindex`

Admin: resets the indexer cursor and rebuilds the event index from block 0.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST`/api/v1/chain/track`

Records and waits for a transaction sent by the signed-in browser wallet, then refreshes projections.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "hash"
  ],
  "properties": {
    "hash": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{64}$"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Other

GET`/api/v1/stream`

Server-Sent Events stream of live updates (telemetry, chain events, jobs, commands...) with Last-Event-ID replay and a 5-second heartbeat.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/overview`

Dashboard counts: assets by condition/lifecycle, open incidents, pending transfers, device health, evidence and job status.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/audit`

Audit log of every operator, oracle and device action (newest first).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET`/api/v1/diagnostics`

Operational state: network URLs for devices, oracle job queue, sessions, authentication failures.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`